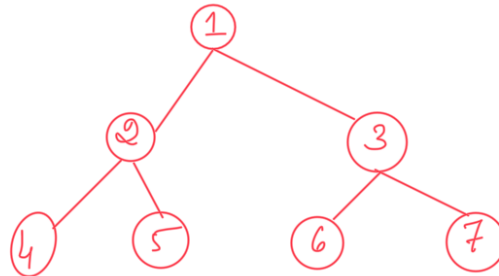


Trees

Day 175

01/01/2026

Problem 8: Give an algorithm for printing the level order data in reverse order. For Example, the output for the below tree should be 4 5 6 7 2 3 1



Solution: Can Be achieved Using Queue & Stack Together

Algorithm (Queue + stack)

- 1 if root == Null return
- 2 Create an empty Queue Q
- 3 Create an empty Stack S
- 4 Enqueue root into Q
- 5 While Q is not Empty
 - Dequeue a node curr from Q
 - Push curr into S
 - if curr.right is not null, Enqueue it into Q.
 - if curr.left is not null, Enqueue it into Q.
- 6 While S is not Empty.
 - pop elements from S and print their data

Key Logic

- Right child is Enqueued Before left
- Stack reverse the order → final output becomes left to right per level.

Code

```
public static void reverseLevelOrder(TreeNode root) {
    if (root == null) return;

    Queue q = new LinkedList<>();
    Stack st = new Stack<>();

    q.add(root);

    while (!q.isEmpty()) {
        TreeNode cur = q.poll();
        st.push(cur);

        // enqueue right first
        if (cur.right != null) q.add(cur.right);
        if (cur.left != null) q.add(cur.left);
    }

    while (!st.isEmpty()) {
        System.out.print(st.pop().data + " ");
    }
}
```

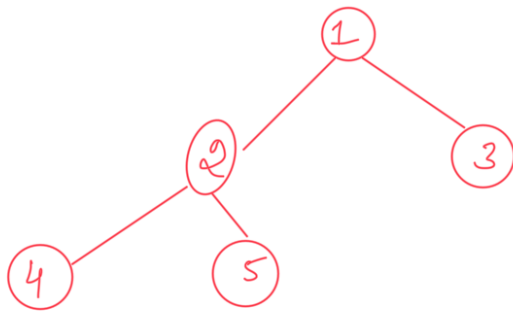
Problem 9 Give an algorithm for deleting a tree

Solution:- To delete a tree, we must traverse all the nodes of the tree and delete them one by one. So which traversal should we use

InOrder PreOrder PostOrder or level Order Traversal?

Before deleting a parent node we should delete its children nodes first. We can use post order traversal as it does the work without storing anything.

We can delete tree with other traversal also with extra space complexity.



The tree nodes are deleted in order

4, 5, 2, 3, 1



Algorithm

- ① if root == null return
- ② Recursively delete the left subtree
- ③ Recursively delete the right subtree
- ④ Delete the current node
- ⑤ Set root == null

Note: Java has garbage collection so we remove references

Code

```
public class DeleteTree {  
  
    public static void deleteTree(TreeNode root) {  
        if (root == null)  
            return;  
  
        // delete left subtree  
        deleteTree(root.left);  
  
        // delete right subtree  
        deleteTree(root.right);  
  
        // delete current node  
        System.out.println("Deleting node: " + root.data);  
        root.left = null;  
        root.right = null;  
        root = null;  
    }  
}
```